

Кто кому доверяет

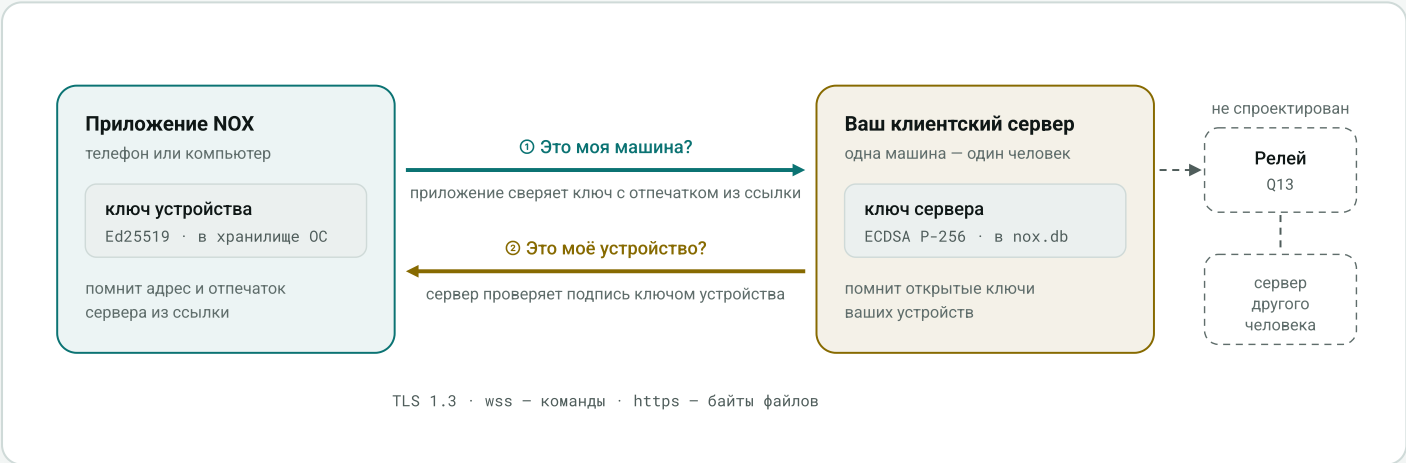
У NOX нет ни удостоверяющего центра, ни паролей. Доверие держится на **двух ключах**, и каждое подключение проверяет оба: приложение — что ответила именно ваша машина, сервер — что пришло именно ваше устройство.

■ приложение, устройство ■ ваш сервер ■ сбой, атака □ ещё не существует

- Общая картина
- Два ключа
- Сертификат
- Каждое подключение
- Ссылка
- Спаривание
- Сбои
- Бэкап
- Детали

Общая картина

При каждом подключении стороны проверяют друг друга: приложение — что ответил **именно его сервер**, сервер — что пришло **именно его устройство**. Выдать себя за одного из них можно, только имея его секретный ключ, а он хранится только у него самого. «Сервер» здесь и дальше — ваш клиентский сервер (client_backend).



Приложение разговаривает только со своим сервером. Всё, что между разными людьми, пойдёт через релей между серверами — его протокола пока нет, это открытый вопрос Q13.

ЗАЧЕМ СВЕРЯТЬ ОТПЕЧАТОК

Допустим, кто-то в той же Wi-Fi-сети выдаёт свою машину за ваш сервер. Соединение с ним тоже будет зашифровано: шифрование прячет переписку от посторонних, но не говорит, с кем вы соединились. Поэтому приложение сверяет ключ ответившей машины с отпечатком из ссылки спаривания. Не совпал — приложение обрывает соединение, ничего не отправив, и показывает, что ответил чужой сервер.

Два ключа — две личности

Оба называются «ключ», но принадлежат разным сторонам и доказывают противоположное.

● Ключ сервера	ECDSA P-256	■ Ключ устройства	Ed25519
Рождается	один раз, при первом старте сервера	Рождается	на устройстве, при первом спаривании
Живёт	внутри файла базы, таблица <code>server_identity</code> . Не зашифрован	Живёт	в защищённом хранилище ОС, <code>session.device_secret</code> . Никогда не покидает устройство
Наружу уходит	открытый ключ — в сертификате при каждом подключении; его отпечаток, sha256, 32 байта, — в ссылке спаривания	Наружу уходит	только открытый ключ — в <code>pair</code> и в каждом приветствии
Доказывает	«я та самая машина, с которой вы спаривались»	Доказывает	«я то самое устройство, которое вы пустили»
Почему P-256	TLS-стек Dart не принимает Ed25519 в рукопожатии	При выходе	стирается; следующий вход — это уже новое устройство

Откуда берётся сертификат

Никто его не выдаёт — сервер собирает его сам из своего ключа.



Пин стоит на ключе, а не на сертификате: сертификат меняется на каждом запуске, ключ — никогда.

- самоподписанный · издатель — он сам
- CN «NOX client server», без SAN
- срок -24 ч ... +100 лет, никто не проверяет
- TLS 1.3 · http/1.1
- на диск не пишется

ПОЧЕМУ НЕ НУЖЕН УДОСТОВЕРЯЮЩИЙ ЦЕНТР

Центр отвечает на вопрос «принадлежит ли этот ключ имени», а у домашнего сервера нет имени, которое стоило бы проверять. Роль центра здесь играет ссылка спаривания: человек сам принёс отпечаток — с машины сервера или с экрана уже спаренного устройства. Поэтому имя, срок и издатель сертификата не проверяются вообще — различает машины только ключ.

Каждое подключение

Четыре шага при каждом соединении — и после рестарта сервера, и после смены сети.

1

TLS и сверка отпечатка

Приложение само проводит рукопожатие и берёт сертификат, которым ответила машина. sha256 его ключа должен совпасть с отпечатком из ссылки. Не совпал — соединение рвётся до первого байта.

2

Вызов

Сервер присылает 32 свежих случайных байта. На каждое соединение — новые, поэтому записанную подпись нельзя повторить.

3

Подпись

Устройство подписывает строку «nox/challenge/v1:» вместе с байтами вызова своим ключом Ed25519 и отправляет session.hello.

4

Проверка и вход

Сервер проверяет подпись и ищет ключ в таблице devices. Нашёл — отвечает курсором и начинает присылать события.

Проверка одинакова для обоих каналов: команды по wss и байты файлов по https идут через один и тот же закреплённый клиент. Какие команды идут по каналу — в NOX Wire Map (nox-wire-map.pdf).

Ссылка спаривания

Три факта в одной строке: куда звонить, кого там ожидать и право войти.

https://nox.app/p/#AQF_AAABH5DzkaMPP0_ZmTJroQhneNuS9XC2MZA_5Du-4GU4er-MHspg6YKBq0BFbpUV470VuiM

после # — 56 байт, 75 символов base64url. Фрагмент браузер не отправляет на сервер nox.app — токен туда не попадает.

11IPv4порт

отпечаток ключа сервера · 32 байта

токен · 16 байт

адрес · 8 байт
куда звонить

отпечаток
кого ожидать: sha256 открытого ключа сервера

токен
право войти, один раз

версия 1 · тип адреса: 1 IPv4 (4 байта), 2 IPv6 (16 байт), 3 DNS (1 байт длины + имя) · порт — 2 байта big-endian

Отпечаток занимает больше половины ссылки — это и есть то, ради чего она существует.

Два вида токена

	CLAIM	ПРИГЛАШЕНИЕ УСТРОЙСТВА
Кто выпускает	сервер при каждом старте, пока у него нет ни одного устройства	уже спаренное устройство: Devices → Add a device
Срок	нет — умирает, когда потрачен	10 минут
Сколько раз	один	один
Что даёт	человек, который спарился первым, становится владельцем сервера. Если на сервере не осталось ни одного устройства — все отозваны или вышли, пока были на связи, — claim возвращает к тому же человеку вместе с историей	ещё одно устройство того же человека
Где увидеть	служебная страница на машине сервера; QR — если сервер доступен из сети	экран Devices, ссылка и QR

Когда
гасится

успешный claim гасит все прочие claim-токены

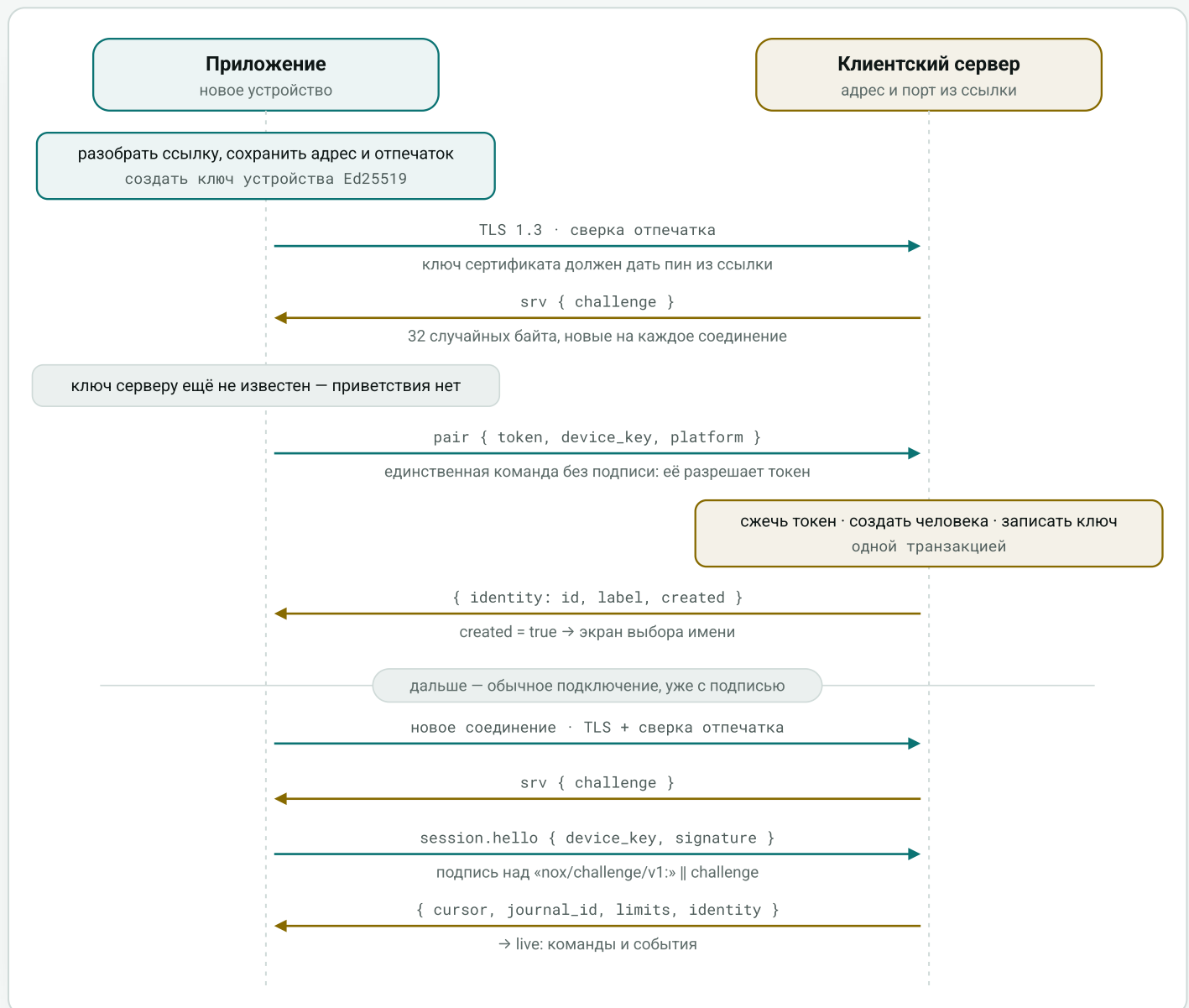
отзыв любого устройства гасит все
живые приглашения этого человека

ГДЕ ВЗЯТЬ ПЕРВУЮ ССЫЛКУ

Claim-ссылку показывает сам сервер — на служебной странице, которая открывается только на его машине. Там же QR, если сервер доступен из локальной сети: телефон сканирует его. Если сервер слушает только свою машину, QR нет — ссылку вставляют в приложение на ней же. Подключаются устройства из той же локальной сети — подробнее в [NOX Wire Map](#) (nox-wire-map.pdf).

Спаривание

Один раз на устройство. Сразу после него — обычное подключение с подписью.



Второе устройство проходит тот же путь по приглашению с уже спаренного:

Devices → Add a device. Разница одна — человек уже есть, поэтому created = false.

Что может пойти не так

Главное различие — стираются ли данные на устройстве.

СИТУАЦИЯ	ЧТО ВИДИТ ЧЕЛОВЕК	ЛОКАЛЬНЫЕ ДАННЫЕ	ВЫХОД
По адресу отвечает чужой ключ	баннер «This isn't the server you paired with» и кнопка «Try again»; отправка ждёт	не стирается	Try again или перезапуск. Если сервер правда сменился — Log out и новая ссылка
При входе ссылка ведёт не к той машине	«This server doesn't match its link»	не стирается	вход откатывается; нужна правильная ссылка
Устройство отозвали с другого устройства	выход, «Your session expired»	стирается	новое приглашение с оставшегося устройства
Сервер не знает ключ устройства	то же, что отзыв: выход, «Your session expired»	стирается	спарить заново. Бывает, если отозвали, пока устройство было офлайн, или базу восстановили из старого бэкапа
База сервера потеряна	на всех устройствах баннер о чужом сервере	не стирается	Log out, затем claim заново; история на сервере потеряна
Сменился адрес сервера	обычное «No connection»	не стирается	вернуть серверу прежний адрес; обновить адрес на устройстве нечем
Приглашению больше 10 минут	«This link has expired. Ask for a new one.»	вход не начался	выпустить новое
Токен потрачен или чужой	«This link can't be used. Ask for a new one.»	вход не начался	новая ссылка
Ссылка повреждена	«This isn't a pairing link»	вход не начался	отсканировать или вставить заново

ПОЧЕМУ ЧУЖОЙ КЛЮЧ НИЧЕГО НЕ СТИРАЕТ

Отказ по сертификату намеренно не ведёт к выходу из аккаунта. Иначе любой, кто встал посередине, мог бы стереть все устройства человека, просто предъявив свой сертификат. Поэтому приложение перестаёт звонить, показывает баннер и ждёт — переписка на месте, никто не разлогинен.

Бэкап и потеря сервера

Файл `nox.db` — это и есть личность сервера: в нём ключ, люди, устройства, токены, чаты и сообщения. Пока сервер работает, свежие записи могут лежать в соседнем `nox.db-wal`, поэтому копируйте базу с остановленного сервера. Байты вложений лежат рядом, в `nox.db-files/`.

■ Восстановлен целиком

Тот же ключ — отпечатки сходятся, устройства подключаются сами. Устройства, спаренные уже после бэкапа, будут разлогинены: их ключей в старой базе нет.

■ Файл потерян

На следующем старте сервер создаст новый ключ, и все устройства увидят чужой сервер. Выход — Log out и новая claim-ссылка. История на сервере потеряна.

■ Восстановлен без ключа

Сервер откажется стартовать. Создавать новый ключ при живых людях он не будет — это молча сломало бы все устройства.

ХРАНИТЕ БЭКАП КАК САМ СЕРВЕР

Кто может прочитать файл, получает и ключ сервера, и всю переписку: ключ хранится незашифрованным, сообщения — открытым текстом, потому что сквозного шифрования пока нет — это открытый вопрос Q1.

Для инженеров

Как это устроено в коде — и почему именно так.

Реализация проверки

Проверяется лист, а не верх цепочки. Приложение само проводит TLS внутри `connectionFactory` (`SecureSocket.startConnect`) и читает `peerCertificate` — сертификат, чей ключ завершил рукопожатие. `badCertificateCallback` получает верх предъявленной цепочки, а туда можно приложить публичный сертификат настоящего сервера поверх своего листа.

DER обходится по позициям. `ServerPin.matches` идёт к седьмому полю `TBSCertificate`. Поиск ключа по байтам был уязвим: копия настоящего SPKI в поле `subject` поддельного сертификата давала нужный отпечаток. SPKI обязан быть ровно 91 байт с заголовком P-256.

Никаких системных корней. `SecurityContext` создаётся без доверенных корней: удостоверяющие центры в решении не участвуют. Ключ и так сверяется в `connectionFactory` на каждом соединении, пустое хранилище лишь убирает лишний источник доверия.

Один клиент на оба транспорта. `WebSocket` и загрузки файлов идут через один `PinnedHttpClient`. Прокси принудительно выключен, `badCertificateCallback` всегда возвращает `false`.

Порядок выхода. Сначала закрывается `WebSocket`, потом `unpin()` забывает отпечаток и закрывает HTTP-клиент.

Проверка не бросает исключений. Любой странный вход — это просто «не совпало», потому что функция работает внутри TLS-стека.

Особый случай. Чужая машина, которая предъявляет публичный сертификат настоящего сервера без его приватного ключа, отсекается самим TLS. Это безопасно, но выглядит как обычный обрыв, а не как баннер о чужом сервере.

Проверить отпечаток сервера руками

```
# должен совпасть с полем fingerprint в строке listening лога сервера
openssl s_client -connect HOST:PORT </dev/null 2>/dev/null \
| openssl x509 -noout -pubkey \
| openssl pkey -pubin -outform DER \
| openssl dgst -sha256 -binary | base64
```

Контрольный вектор подписи

```
seed      = 0x00 0x01 0x02 ... 0x1f          (32 байта)
public    = A6EHv/POEL4dcN0Y50vAmWfk1jCbpQ1fHdyGZBJVMbg=
message   = "nox/challenge/v1:probe"
signature = puHNSjLy6sqx5MEgiAuZiz6Zuyjohrz0EIEjkdQ9iazgyIEeUvGbK+UAIhHjpscFPCMBN7rpbfeoXr1UZXQZBQ==
```

Закреплён в контракте §2. Проверяется Dart-тестом; на стороне Go тест пока не написан.